Activity 5: Git Mastery & Troubleshooting Activity

Course: CS223 – Application Development and Emerging Technologies

Name: Jhulius Pacis

Section: BSIS-3C

GitHub Username: Julio989898

Repository: https://github.com/Julio989898/Project-Nova

Instructor added as collaborator: madzamador

Part 1: Core Repository Management

Scenario / Task	Git Command
1. Initialization: Initialize a new, empty Git repository in the Project-Nova directory.	`git init`
2. Staging: Create a file named auth-service.js and add it to the staging area.	`git add auth-service.js`
3. Committing: Commit the staged file with the message "Added initial auth service logic."	`git commit -m "Added initial auth service logic."`
4. Branching: Create a new local branch named feature/payment-gateway (without switching to it).	`git branch feature/payment-gateway`

5. Navigation: Switch to the feature/payment-gateway branch.	`git checkout feature/payment-gateway`
6. Publishing: Push the new branch to GitHub and set it to track origin/feature/payment-gateway.	`git push --set-upstream origin feature/payment-gateway`
7. Syncing: Download and integrate the remote changes made by a collaborator.	`git pull origin main`
8. Conflict Check: View the status to see which files have unmerged conflicts.	`git status`
9. Resolution: After manually fixing config.js and staging it, conclude the merge by saving the timeline.	`git add config.js` then `git commit -m "Resolved conflict in config.js"`

Other commands used to set up and verify the repository

Purpose	Git Command
Avoid the divergent-branches warning when pulling	`git config pull.rebase false`
Rename the default branch to main	`git branch -M main`

Link the local repository to GitHub	`git remote add origin https://github.com/Julio989898/Project-Nova.git`
Push main and set up tracking	`git push -u origin main`
Create a base configuration file so a conflict could be simulated	`git add config.js` then `git commit -m "Added base configuration"`
Simulate the collaborator's change on main	`git checkout main`, then `git commit -m "Set production mode in config"` and `git push origin main`
Commit the conflicting change on the feature branch	`git commit -m "Added payment gateway configuration"`
Merge the feature branch into main	`git checkout main` then `git merge feature/payment-gateway`
Publish main	`git push origin main`
View the commit network in the terminal	`git log --oneline --graph --all`

Part 2: Troubleshooting & Emergency Fixes

1. The "Oops, wrong file" Problem

Solution: Unstage the specific file while keeping local changes intact.

Command: `git restore --staged .env` (or `git reset HEAD .env`)

2. The "Typo in the message" Problem

Solution: Modify the most recent commit message without creating a new commit.

Command: `git commit --amend -m "Fixed authentication typo"`

3. The "Forgot a file" Problem

Solution: Stage the forgotten file, then merge it into the previous commit without changing the message.

Command: `git add styles.css` then `git commit --amend --no-edit`

4. The "I hate these changes" Problem

Solution: Discard all local changes in the working directory for that file.

Command: `git restore index.html` (or `git checkout -- index.html`)

5. The "Wrong Branch" Problem

Solution: Create a new branch pointing to the commits, then roll main back by 3 commits.

Command: `git branch new-feature`, then `git reset HEAD~3 --hard`, then `git checkout new-feature`

6. The "Detached HEAD" Problem

Solution: Create a new branch right where you are to save the floating commits.

Command: `git switch -c recovery-branch` (or `git checkout -b recovery-branch`)

7. The "Undo a pushed mistake" Problem

Solution: Create a new commit that does the exact opposite of the buggy commit, keeping the timeline intact.

Command: `git revert <buggy-commit-hash>`

8. The "Accidental Deletion" Problem

Solution: Recover the branch using its last known commit hash, found with git reflog.

Command: `git reflog`, then `git branch <deleted-branch-name> <commit-hash>`

9. The "Ghost Files" Problem

Solution: Tell Git to record the deletions and stage them for the next commit.

Command: `git add -u` (or `git rm <filename>`)

10. The "Pushed to the void" Problem

Solution: Delete the remote branch directly from the terminal.

Command: `git push origin --delete testing-junk`

Network Graph (git log --oneline --graph --all)

The feature/payment-gateway branch splits from main after the commit "Added base configuration" and rejoins at the merge commit "Resolved conflict in config.js". Main, origin/main, and the feature branch point to the same final commit, so the local and remote repositories are synchronized.